

SN 2022-2023

February 15, 2025

1 Exercice 1

```
[6]: # Question 1: Suite de Syracuse
def syracuse(u):
    i = 0
    while u != 1:
        if u % 2 == 0:
            u /= 2
        else:
            u = (u * 3) + 1
        i += 1
    return i

print(syracuse(81))
```

22

```
[8]: # Question 2: Nombres premier tronquables
def est_premier(n):
    """Teste si un nombre est premier."""
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def est_troncable_gauche(n):
    """Teste si un nombre est tronquable de gauche à droite."""
    while n > 0:
        if not est_premier(n):
            return False
        n //= 10
    return True

def est_troncable_droite(n):
    """Teste si un nombre est tronquable de droite à gauche."""
    while n > 0:
```

```

        if not est_premier(n):
            return False
        n %= 10**(len(str(n)) - 1)
    return True

def trouver_nombres_premiers_tronquables():
    """Trouve les nombres premiers tronquables et calcule leur somme."""
    somme = 0
    nombres_trouves = []
    nombre = 10 # Commencer après 7

    while len(nombres_trouves) < 11:
        if est_premier(nombre) and est_troncable_gauche(nombre) and
↪est_troncable_droite(nombre):
            somme += nombre
            nombres_trouves.append(nombre)
            nombre += 1
    return somme, nombres_trouves

somme_totale, nombres_premiers = trouver_nombres_premiers_tronquables()
print("Somme des 11 nombres premiers tronquables :", somme_totale)
print("Nombres premiers tronquables trouvés :", nombres_premiers)

```

Somme des 11 nombres premiers tronquables : 748317

Nombres premiers tronquables trouvés : [23, 37, 53, 73, 313, 317, 373, 797, 3137, 3797, 739397]

```

[16]: # Question 3: Diatomique de Stern
def diatomique_stern(max_n):
    i = 2
    s = [0, 1]
    while i <= max_n:
        if i % 2 == 0:
            s.append(s[i//2])
        else:
            s.append(s[(i-1) // 2] + s[((i-1) // 2) + 1])
        i += 1
    return s[-1]

diatomique_stern(10000001)

```

[16]: 9469

2 Exercice 2

[17]: *# Question 1: définir la fonction NewtonZ*

```
def NewtonZ(f, f_prime, x_0, n):
    x = x_0
    for _ in range(0, n):
        x -= (f(x) / f_prime(x))

    return x
```

[31]: *# Question 2: définir la suite pour la fonction*

```
def f(x):
    return x**2 - 2

def f_prime(x):
    return x * 2

print(f"À 100 itération de la fonction on a : {NewtonZ(f, f_prime, 1, 100)}")

# Comparer avec la fonction interne de python
import math
print(f"La fonction interne de python retourne {math.sqrt(2)}")
```

À 100 itération de la fonction on a : 1.414213562373095

La fonction interne de python retourne 1.4142135623730951

3 Exercice 3

[34]: *# Question 1: Implémenter la fonction Simpson*

```
def Simpson(f, a, b):
    if not a < b:
        return None

    return (b - a) / 6 * (f(a) + 4 * f((a+b) / 2) + f(b))
```

[35]: *# Question 2: Écrire une fonction qui implémente l'équation (2)*

```
def SimpsonComposite(f, a, b, n):
    if not a < b or not isinstance(n, int):
        return None
    h = (b - a) / n
    result = 0
    for k in range(0, n):
        result += f(a + k * h) + 4 * f(a + (k + 1/2) * h) + f(a + (k + 1) * h)
    result *= (h/6)

    return result
```

```
[36]: # Question 3; Calcul des valeurs approché de I1 et I2

import math

def f1(x):
    return math.cos(1 + math.exp(x**2))

def f2(x):
    return math.sin(1 + x**2) / (math.exp(x**2) + 2*math.cos(x) + math.tan(x)**2)

n = 10

# Pour I1
I1_approx = SimpsonComposite(f1, 0, math.pi, n)
print("Valeur approchée de I1 (n=10) :", I1_approx)

# Pour I2
I2_approx = SimpsonComposite(f2, 0, 1, n)
print("Valeur approchée de I2 (n=10) :", I2_approx)
```

Valeur approchée de I1 (n=10) : -0.6841543736638913
 Valeur approchée de I2 (n=10) : 0.2592714726317215

[]: